

EE2016: Microprocessor

Experiment 3: Interrupts in Atmel AVR Atmega through Assembly Programming

Name: Shitheeksh H V

Roll No: EE24B063

19th August 2025

Aim:

Using Atmel AVR assembly language programming, implement interrupts and DIP switches control in Atmel Atmega microprocessor. Aims of this experiment are:

- (i) Generate an external (logical) hardware interrupt using an emulation of a push button switch.
- (ii) Write an ISR (Interrupt Service Routine) to switch ON an LED for a few seconds (10 secs) and then switch OFF. (The lighting of the LED could be verified by monitoring the signal to switch it ON).
- (iii) If there is time, you could try this also: Use the 16 bit timer (via interrupt) to make an LED blink with a duration of 1 second. Also, one needs to implement all of the above, in AVR assembly.

Tasks:

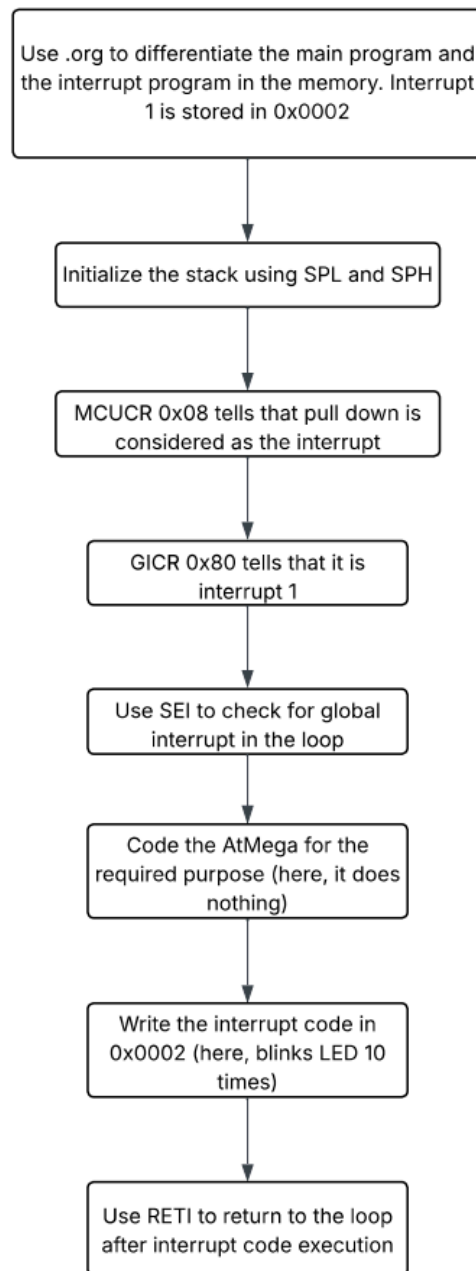
Task 1: Fill in the blanks in the assembly code

Task 2: Use `int0` to redo the same in the demo program (duly filled in). Once the switch is pressed the LED should blink 10 times (ON (or OFF) - 1 sec, duty cycle could be 50 %)

Task 3: Demonstrate the working using assembly program

Task 1:

Flowchart:



Code:

```
;initializing the location of program code
.org 0
rjmp reset

.org 0x0002
rjmp int1_ISR

.org 0x0100

reset:
    ;stack pointers
    LDI R16, 0x70
    OUT SPL, R16
    LDI R16, 0x00
    OUT SPH, R16

    LDI R16, 0x01
    OUT DDRB, R16

    LDI R16, 0x00
    OUT DDRD, R16

    IN R16, MCUCR;    Load MCUCR register
    ORI R16, 0x08
    OUT MCUCR, R16

    IN R16, GICR;    Load GICR register
    ORI R16, 0x80
    OUT GICR, R16

    LDI R16, 0x00
    OUT PORTB, R16

    SEI; start global interrupt check
ind_loop:rjmp ind_loop

int1_ISR:IN R16,SREG
    PUSH R16

    LDI R16, 0x0A
    MOV R0, R16

    ;time delay
c1: LDI R16, 0x01
    OUT PORTB, R16
```

```
LDI R16 ,0xFF
a1: LDI R17 ,0xFF
a2: DEC R17
    BRNE a2
    DEC R16
    BRNE a1

    LDI R16,0x00
    OUT PORTB,R16

    LDI R16,0xFF
b1: LDI R17,0xFF
b2: DEC R17
    BRNE b2
    DEC R16
    BRNE b1

    DEC R0
    BRNE c1
    POP R16
    OUT SREG,R16

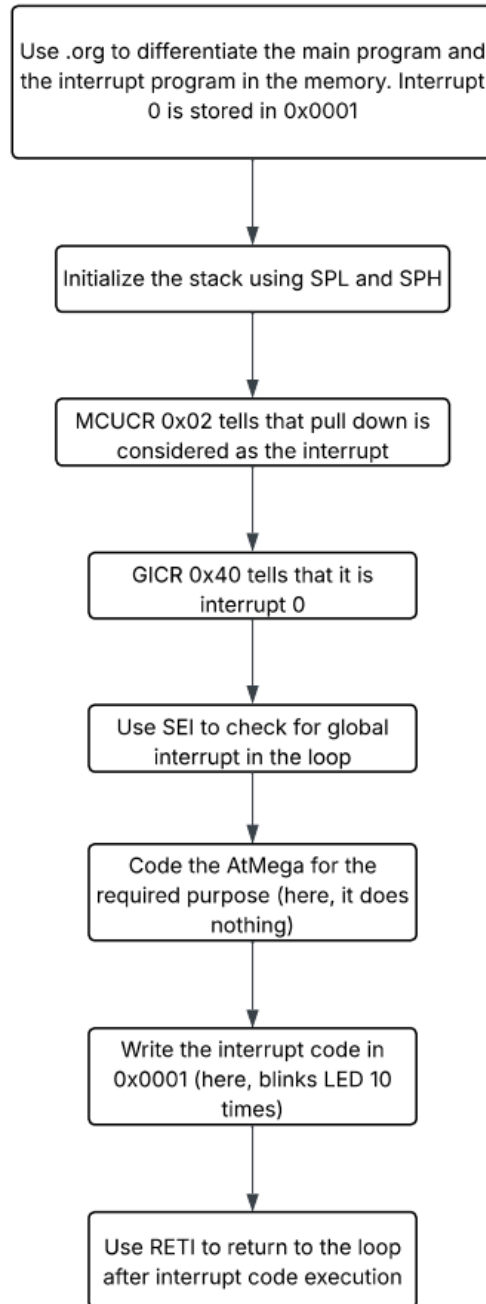
    RETI
```

Output:

https://drive.google.com/file/d/1hGqiFKgeGA1o8QxRfkzWiFbJCbGUbvS_/view?usp=drive_link

Task 2:

Flow Chart:



Code:

```
;initializing the location of program code
.org 0
rjmp reset

.org 0x0001
rjmp int0_ISR

.org 0x0100

reset:
    ;stack pointers
    LDI R16, 0x70
    OUT SPL, R16
    LDI R16, 0x00
    OUT SPH, R16

    LDI R16, 0x01
    OUT DDRB, R16

    LDI R16, 0x00
    OUT DDRD, R16

    IN R16, MCUCR;    Load MCUCR register
    ORI R16, 0x02
    OUT MCUCR, R16

    IN R16, GICR;    Load GICR register
    ORI R16, 0x40
    OUT GICR, R16

    LDI R16, 0x00
    OUT PORTB, R16

    SEI; start global interrupt check
ind_loop:rjmp ind_loop

int0_ISR:IN R16,SREG
    PUSH R16

    LDI R16, 0x0A
    MOV R0, R16

    ;time delay
c1: LDI R16, 0x01
    OUT PORTB, R16
```

```
LDI R16 ,0xFF
a1: LDI R17 ,0xFF
a2: DEC R17
    BRNE a2
    DEC R16
    BRNE a1
```

```
LDI R16,0x00
OUT PORTB,R16
```

```
LDI R16,0xFF
b1: LDI R17,0xFF
b2: DEC R17
    BRNE b2
    DEC R16
    BRNE b1
```

```
DEC R0
BRNE c1
POP R16
OUT SREG,R16
```

```
RETI
```

Output:

https://drive.google.com/file/d/11uTmUWXEwjUODXTHwca6t0pdxeZjlsJ8/view?usp=drive_link